DAY 9: MASTERING PROBABILITY WITH PYTHON

Date: May 21, 2026 **Student Name:** Abu Fraz Abbas

Objective: Learning Custom Probability Engines (Manual Logic) and Standard Library Solutions (Built-in Modules) for real-world auditing, business, and inventory scenarios.

Condition 1: Basic Master Formula & Complementary Rule

The core definition of probability is the ratio of favorable outcomes to the total outcomes in a sample space. The Complementary Rule states that the probability of an event NOT happening is 1 minus the probability of that event happening.

> ***P(E) = Favorable Outcomes / Total Outcomes***
> ***P(E') = 1 - P(E)***

1. Manual Python Code

```
def calculate_basic_probability(favorable_outcomes, total_outcomes):
    if total_outcomes <= 0:
        return "Total outcomes must be greater than 0."
    if favorable_outcomes > total_outcomes:
        return "Favorable outcomes cannot exceed total outcomes."

    probability_event = favorable_outcomes / total_outcomes
    probability_complement = 1 - probability_event

    return {
        "Probability of Event P(E)": round(probability_event, 4),
        "Probability Percentage": f"{round(probability_event * 100, 2)}%",
        "Complement Probability P(E')": round(probability_complement, 4),
        "Complement Percentage": f"{round(probability_complement * 100, 2)}%"
    }

# Q1: Diamond Rings (4 Diamond out of 12 Total)
print(calculate_basic_probability(4, 12))
# Q2: Ledger Page Audit (35 No-Error Pages out of 50 Total)
print(calculate_basic_probability(35, 50))
```

2. Short Library Code

```
from fractions import Fraction

def calculate_builtin_probability(favorable, total):
    exact_fraction = Fraction(favorable, total)
    return {
        "Fraction Form": exact_fraction,
        "Decimal Value": round(float(exact_fraction), 4),
        "Percentage Form": f"{round(float(exact_fraction) * 100, 2)}%",
        "Complement Fraction": 1 - exact_fraction
    }
```

Condition 2: Addition Rule - Mutually Exclusive Events ("OR" Logic)

Mutually Exclusive events are events that cannot occur at the same time. If one happens, the other cannot. In these scenarios, we add the individual probabilities together.

> $P(A \text{ or } B) = P(A) + P(B)$

1. Manual Python Code

```
def calculate_addition_exclusive(prob_A, prob_B):
    if not (0 <= prob_A <= 1) or not (0 <= prob_B <= 1):
        return "Error: Individual probabilities must be between 0 and 1."
    if (prob_A + prob_B) > 1:
        return "Error: Sum of mutually exclusive probabilities cannot exceed 100%."

    total_probability = prob_A + prob_B
    return {
        "P(A)": round(prob_A, 4),
        "P(B)": round(prob_B, 4),
        "Total P(A or B)": round(total_probability, 4),
        "Total Percentage": f"{round(total_probability * 100, 2)}%"
    }

# Q3: Ludo Dice (Even Number [3/6] OR Number 5 [1/6])
print(calculate_addition_exclusive(3/6, 1/6))
# Q4: Customer Choice (Chain [0.25] OR Bangles [0.15])
print(calculate_addition_exclusive(0.25, 0.15))
```

2. Short Library Code

```
from fractions import Fraction

def short_addition_rule(favorable_A, favorable_B, total):
    total_prob = Fraction(favorable_A, total) + Fraction(favorable_B, total)
    return {"Total P(A or B)": total_prob, "Percentage": f"{round(float(total_prob) * 100,
2)}%"}
```

Condition 3: Addition Rule with Overlap (Non-Mutually Exclusive)

When two events can happen simultaneously, they share an overlapping intersection. To avoid double-counting, the common section must be subtracted once.

> $P(A \text{ or } B) = P(A) + P(B) - P(A \cap B)$

1. Manual Python Code

```
def calculate_addition_non_exclusive(prob_A, prob_B, prob_both):
    if prob_both > prob_A or prob_both > prob_B:
        return "Error: Common probability cannot be greater than individual
probabilities."

    total_probability = prob_A + prob_B - prob_both
    return {
        "Total Probability P(A or B)": round(total_probability, 4),
        "Total Percentage": f"{round(total_probability * 100, 2)}%"
    }

# Q5: Playing Cards (Black Card [26/52] OR King [4/52] - Common Black Kings [2/52])
print(calculate_addition_non_exclusive(26/52, 4/52, 2/52))
# Q6: Inventory Audit Overlap (Tag Error [0.3] OR Weight Mismatch [0.2] - Both [0.1])
print(calculate_addition_non_exclusive(0.3, 0.2, 0.1))
```

2. Short Library Code

```
from fractions import Fraction

def short_non_exclusive_rule(fav_A, fav_B, fav_both, total):
    total_prob = Fraction(fav_A, total) + Fraction(fav_B, total) - Fraction(fav_both,
total)
    return {"Total P(A or B)": total_prob, "Percentage": f"{round(float(total_prob) * 100,
2)}%"}
```

Condition 4: Multiplication Rule - Independent Events ("AND" Logic)

Independent events are events where the outcome of the first event has absolutely no influence on the second event. When looking for compound probability (A AND B), we multiply their chances.

> $P(A \text{ and } B) = P(A) \times P(B)$

1. Manual Python Code

```
def calculate_multiplication_independent(prob_A, prob_B):
    total_probability = prob_A * prob_B
    return {
        "Combined Probability P(A and B)": round(total_probability, 4),
        "Total Percentage": f"{round(total_probability * 100, 2)}%"
    }

# Q7: Coin Head (1/2) AND Dice Prime Number (3/6)
print(calculate_multiplication_independent(1/2, 3/6))
# Q8: Factory Machine Defects (Machine 1 [0.05] AND Machine 2 [0.10])
print(calculate_multiplication_independent(0.05, 0.10))
```

2. Short Library Code

```
from fractions import Fraction

def short_multiplication_independent(fav_A, total_A, fav_B, total_B):
    total_prob = Fraction(fav_A, total_A) * Fraction(fav_B, total_B)
    return {"Total P(A and B)": total_prob, "Percentage": f"{round(float(total_prob) *
100, 2)}%"}
```

Condition 5: Conditional Probability Rule (Given Context Matrix)

Conditional Probability calculates the probability of an event happening, given that another event has already occurred. This restricts the sample space to the known condition pool.

> $P(B|A) = P(A \cap B) / P(A)$

1. Manual Python Code

```
def calculate_conditional_probability(prob_A, prob_both):
    if prob_A == 0:
        return "Error: Given condition probability cannot be zero."

    conditional_prob = prob_both / prob_A
    return {
        "Conditional Probability P(B|A)": round(conditional_prob, 4),
        "Total Percentage": f"{round(conditional_prob * 100, 2)}%"
    }

# Q9: GST Tax Audit (Purchase Wrong [0.40], Both Wrong [0.15])
print(calculate_conditional_probability(0.40, 0.15))
```

2. Short Library Code

```
from fractions import Fraction

def short_conditional_probability(fav_both, fav_A):
    total_prob = Fraction(fav_both, fav_A)
    return {"Conditional P(B|A)": total_prob, "Percentage": f"{round(float(total_prob) *
100, 2)}%"}

# Q10: Lost Card King Given It Is Red (2 Red Kings out of 26 Red Cards)
print(short_conditional_probability(2, 26))
```

Condition 6: Multiplication Theorem for Dependent Events ("Without Replacement")

In selections performed "Without Replacement", both the favorable pool and the total available items pool decrement by exactly 1 for the consecutive trial.

> $P(A \text{ and } B) = P(A) \times P(B|A)$

1. Manual Python Code

```
def calculate_multiplication_dependent(favorable, total):
    prob_A = favorable / total

    fav_after_first = favorable - 1
    total_after_first = total - 1
    prob_B_given_A = fav_after_first / total_after_first

    total_probability = prob_A * prob_B_given_A
    return {
        "Combined Probability (Both)": round(total_probability, 4),
        "Total Percentage": f"{round(total_probability * 100, 2)}%"
    }

# Q11: Continuous Gold Coin Extraction (6 24K out of 10 Total Coins)
print(calculate_multiplication_dependent(6, 10))
# Q12: Defective Laptop Inspection (2 Defective out of 8 Total Laptops)
print(calculate_multiplication_dependent(2, 8))
```

2. Short Library Code

```
from fractions import Fraction

def short_multiplication_dependent(fav, total):
    total_prob = Fraction(fav, total) * Fraction(fav - 1, total - 1)
    return {"Total P(Both)": total_prob, "Percentage": f"{round(float(total_prob) * 100,
2)}%"}
```

End of Day 9 Study Guide Document. Python Scripts verified and outputs cataloged.